2
Fundamental algorithms

We start by discussing the computer representation and fundamental arithmetic algorithms for integers and polynomials. We will keep this discussion fairly informal and avoid all the intricacies of actual computer arithmetic—that is a topic on its own. The reader must be warned that modern-day processors do *not* represent numbers and operate on them as we describe now, but to describe the tricks they use would detract us from our current goal: a simple description of how one *could*, in principle, perform basic arithmetic.

Although our straightforward approach can be improved in practice for arithmetic on small objects, say double-precision integers, it is quite appropriate for large objects, at least as a start. Much of this book deals with polynomials, and we will use some of the notions of this chapter throughout. A major goal is to find algorithmic improvements for large objects.

The algorithms in this chapter will be familiar to the reader, but she can refresh her memory of the *analysis of algorithms* with our simple examples.

2.1. Representation and addition of numbers

Algebra starts with numbers and computers work on data, so the very first issue in computer algebra is how to feed numbers as data into computers. Data are stored in pieces called **words**. Current machines use either 32- or 64-bit words; to be specific, we assume that we have a 64-bit processor. Then one machine word contains a **single precision** integer between 0 and $2^{64}-1$.

How can we represent integers outside the range $\{0,\dots,2^{64}-1\}$? Such a **multiprecision integer** is represented by an array of 64-bit words, where the first one encodes the sign of the integer and the length of the array. To be precise, we consider the 2^{64}-ary (or radix 2^{64}) representation of a nonzero integer

$$a = (-1)^s \sum_{0\le i\le n} a_i \cdot 2^{64i}, \tag{1}$$

where $s \in \{0,1\}$, $0 \le n+1 < 2^{63}$, and $a_i \in \{0,\dots,2^{64}-1\}$ for all i are the **digits**

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

(in base 2^{64}) of a. We encode it as an array

$$s \cdot 2^{63} + n + 1, a_0, \ldots, a_n$$

of 64-bit words. This representation can be made unique by requiring that the **leading digit** a_n be nonzero if $a \neq 0$ (and using the single-entry array 0 to represent $a = 0$). We will call this the **standard representation** for a. For example, the standard representation of -1 is $2^{63} + 1, 1$. It is, however, convenient also to allow nonstandard representations with leading zero digits since this sometimes facilitates memory management, but we do not want to go into details here. The range of integers that can be represented in standard representation on a 64-bit processor is between $-2^{64 \cdot 2^{63}} + 1$ and $2^{64 \cdot 2^{63}} - 1$; each of the two boundaries requires $2^{63} + 1$ words of storage. This size limitation is quite sufficient for practical purposes: one of the larger representable numbers would fill about 70 million 1-TB-discs.

For a nonzero integer $a \in \mathbb{Z}$, we define the **length** $\lambda(a)$ of a as

$$\lambda(a) = \lfloor \log_{2^{64}} |a| \rfloor + 1 = \left\lfloor \frac{\log_2 |a|}{64} \right\rfloor + 1,$$

where $\lfloor \cdot \rfloor$ denotes rounding down to the nearest integer (so that $\lfloor 2.7 \rfloor = 2$ and $\lfloor -2.7 \rfloor = -3$). Thus $\lambda(a) + 1 = n + 2$ is the number of words in the standard representation (1) of a (see Exercise 2.1). This is quite a cluttered expression, and it is usually sufficient to know that about $\frac{1}{64} \log_2 |a|$ words are needed, or even more succinctly $O(\log_2 |a|)$, where the **big-Oh notation** "O" hides an arbitrary constant (Section 25.7).

We assume that our hypothetical processor has at its disposal a command for the addition of two single precision integers a and b. The output of the addition command is a 64-bit word c plus the content of the **carry flag** $\gamma \in \{0, 1\}$, a special bit in the processor status word which indicates whether the result exceeds 2^{64} or not. In order to be able to perform addition of multiprecision integers more easily, the carry flag is also *input* to the addition command. More precisely, we have

$$a + b + \gamma = \gamma^* \cdot 2^{64} + c,$$

where γ is the value of the carry flag before the addition and γ^* is its value afterwards. Usually there are processor instructions to clear and set the carry flag.

If $a = \sum_{0 \leq i \leq n} a_i 2^{64i}$ and $b = \sum_{0 \leq i \leq m} b_i 2^{64i}$ are two multiprecision integers, then their sum is

$$c = \sum_{0 \leq i \leq k} (a_i + b_i) 2^{64i},$$

where $k = \max\{n, m\}$, and if, say, $m \leq n$, then $b_{m+1}, \ldots, b_n$ are set to zero. (In other words, we may assume that $m = n$.) In general, $a_i + b_i$ may be larger than 2^{64}, and if so, then the carry has to be added to the next digit in order to get a 2^{64}-ary

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

representation again. This process propagates from the lower order to the higher order digits, and in the worst case, a carry from the addition of a_0 and b_0 may influence the addition of a_n and b_n, as the example $a = 2^{64(n+1)} - 1$ and $b = 1$ shows. Here is an algorithm for the addition of two multiprecision integers of the same sign; see Exercise 2.3 for a subtraction algorithm.

ALGORITHM 2.1 Addition of multiprecision integers.
Input: Multiprecision integers $a = (-1)^s \sum_{0\le i\le n} a_i 2^{64i}$, $b = (-1)^s \sum_{0\le i\le n} b_i 2^{64i}$, not necessarily in standard representation, with $s \in \{0,1\}$.
Output: The multiprecision integer $c = (-1)^s \sum_{0\le i\le n+1} c_i 2^{64i}$ such that $c = a + b$.

1. $\gamma_0 \longleftarrow 0$
2. **for** $i = 0, \ldots, n$ **do**
   $c_i \longleftarrow a_i + b_i + \gamma_i, \quad \gamma_{i+1} \longleftarrow 0$
   if $c_i \ge 2^{64}$ **then** $c_i \longleftarrow c_i - 2^{64}, \quad \gamma_{i+1} \longleftarrow 1$
3. $c_{n+1} \longleftarrow \gamma_{n+1}$
   return $(-1)^s \sum_{0\le i\le n+1} c_i 2^{64i}$

We use the sign $\longleftarrow$ in algorithms to denote an assignment, and indentation distinguishes the body of a loop.

We practise this algorithm on the addition of $a = 9438 = 9 \cdot 10^3 + 4 \cdot 10^2 + 3 \cdot 10 + 8$ and $b = 945 = 9 \cdot 10^2 + 4 \cdot 10 + 5$ and in decimal (instead of 2^{64}-ary) representation:

i	4	3	2	1	0
a_i		9	4	3	8
b_i		0	9	4	5
γ_i	1	1	0	1	0
c_i	1	0	3	8	3

How much time does this algorithm use? The basic subroutine, the addition of two single precision integers, requires some number of machine cycles, say k. This number will depend on the processor used, and the actual CPU time depends on the processor's speed. Thus the addition of two integers of length at most n takes kn machine cycles for single precision additions, plus some number of cycles for control structures, manipulating sign bits, index arithmetic, memory access, etc. We will, however, (correctly) assume that the number of machine cycles for the latter has the same order of magnitude as the cost for the single precision arithmetic operations that we count.

For the remainder of this text, it is vital to abstract from machine-dependent details as discussed above. We will then just say that the addition of two n-word

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

integers can be done in time $O(n)$, or at cost $O(n)$, or with $O(n)$ **word operations**; the constants hidden in the big-Oh will depend on the details of the machine. We gain two advantages from this concept: a shorter and more intuitive notation, and independence of particular machines. The abstraction is justified by the fact that the *actual* performance of an algorithm often depends on compiler optimization, clever cache usage, pipelining effects, and many other things that are quite technical and nearly impossible to describe in a comparatively high-level programming language. However, experiments show that "big-Oh" statements are reflected surprisingly well by implementations on any kind of sequentially working processor: adding two multiprecision integers is a linear operation in the sense that doubling the input size also approximately doubles the running time.

One can make these statements more precise and formally satisfying. The cost measure that is widely used for algorithms dealing with integers is the number of **bit operations** which can be rigorously defined as the number of steps of a Turing or register machine (random access machine, RAM) or the number of gates of a Boolean circuit implementing the algorithm. Since the details of those computational models are rather technical, however, we will content ourselves with informal arguments and cost measures, as above.

Related data types occurring in currently available processors and mathematical software are single and multiprecision **floating point numbers**. These represent approximations of real numbers, and arithmetic operations, such as addition and multiplication, are subject to **rounding errors**, in contrast to the arithmetic operations on multiprecision integers, which are **exact**. Algorithms based on computations with floating point numbers are the main topic in *numerical analysis*, which is a theme of its own; neither it nor the recent attempts at systematically combining exact and numerical computations will be discussed in this text.

2.2. Representation and addition of polynomials

The two main data types on which our algorithms operate are **numbers** as above and **polynomials**, such as $a = 5x^3 - 4x + 3 \in \mathbb{Z}[x]$. In general, we have a commutative **ring** R, such as $\mathbb{Z}$, in which we can perform the operations of addition, subtraction, and multiplication according to the usual rules; see Section 25.2 for details. (All our rings have a multiplicative unit element 1.) If we can also divide by any nonzero element, as in the rational numbers $\mathbb{Q}$, then R is a **field**.

A polynomial $a \in R[x]$ in x over R is a finite sequence $(a_0, \ldots, a_n)$ of elements of R (the **coefficients** of a), for some $n \in \mathbb{N}$, and we write it as

$$a = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = \sum_{0 \le i \le n} a_i x^i. \tag{2}$$

If $a_n \neq 0$, then $n = \deg a$ is the **degree** of a, and $a_n = \mathrm{lc}(a)$ is its **leading coefficient**. If $\mathrm{lc}(a) = 1$, then a is **monic**. It is convenient to take $-\infty$ as the degree of the zero

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

polynomial. We can represent a by an array whose ith element is a_i (in analogy to the integer case, we would also need some storage for the degree, but we will neglect this). This assumes that we already have a way of representing coefficients from R. The length (the number of ring elements) of this representation is $n+1$.

For an integer $r \in \mathbb{N}_{>1}$ (in particular, for $r = 2^{64}$ as in the previous section), the representations (2) of a polynomial and the radix r representation

$$a = a_n r^n + a_{n-1} r^{n-1} + \cdots + a_1 r + a_0 = \sum_{0 \le i \le n} a_i r^i,$$

with digits $a_0, \ldots, a_n \in \{0, \ldots, r-1\}$, of an integer a are quite similar. This is particularly visible if we take polynomials over $R = \mathbb{Z}_r = \{0, \ldots, r-1\}$, the ring of integers modulo r, with addition and multiplication modulo r (Sections 4.1 and 25.2). This similarity is an important point for computer algebra; many of our algorithms apply (with small modifications) to both the integer and polynomial cases: multiplication, division with remainder, gcd and Chinese remainder computation. It is also relevant to note where this does not apply: the subresultant theory (Chapter 6) and, most importantly, the factorization problem (Parts III and IV). At the heart of this distinction lies the deceptively simple carry rule. It gives the low digits some influence on the high digits in addition of integers, and messes up the cleanly separated rules in the addition of two polynomials

$$a = \sum_{0 \le i \le n} a_i x^i \text{ and } b = \sum_{0 \le i \le m} b_i x^i \tag{3}$$

in $R[x]$. This is quite easy:

$$c = a + b = \sum_{0 \le i \le n} (a_i + b_i) x^i = \sum_{0 \le i \le n} c_i x^i,$$

where the addition $c_i = a_i + b_i$ is performed in R and, as with integers, we may assume that $m = n$.

For example, addition of the polynomials $a = 9x^3 + 4x^2 + 3x + 8$ and $b = 9x^2 + 4x + 5$ in $\mathbb{Z}[x]$ works as follows:

i	3	2	1	0
a_i	9	4	3	8
b_i	0	9	4	5
c_i	9	13	7	13

Here is the (rather trivial) algorithm in our formalism.

ALGORITHM 2.2 Addition of polynomials.
Input: $a = \sum_{0 \le i \le n} a_i x^i$, $b = \sum_{0 \le i \le n} b_i x^i$ in $R[x]$, where R is a ring.
Output: The coefficients of $c = a + b \in R[x]$.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

1. **for** $i = 0, \ldots, n$ **do** $c_i \longleftarrow a_i + b_i$
2. **return** $c = \sum_{0 \le i \le n} c_i x^i$

It is somewhat simpler than integer addition, with its carries. This simplicity propagates down the line for more complicated algorithms such as multiplication, division with remainder, etc. Although integers are more intuitive (we learn about them at a much earlier stage in life), their algorithms are a bit more involved, and we adopt in this book as a general program the strategy to present mainly the simpler polynomial case which allows us to concentrate on the essentials, and often leave details in the integer case to the exercises.

As a first example, we have seen that addition of two polynomials of degree up to n takes at most $n+1$ or $O(n)$ arithmetic operations in R; there is no concern with machine details here. This is a much coarser cost measure than the number of word operations for integers. If, for example, $R = \mathbb{Z}$ and the coefficients are less than B in absolute value, then the cost in word operations is $O(n \log B)$, which is the same order of magnitude as the input size. Moreover, additive operations $+, -$ in R are counted at the same cost as multiplicative operations $\cdot, /$, while in most applications the latter are significantly more expensive than the former.

As a general rule, we will analyze the number of **arithmetic operations** in the ring R (additions and multiplications, and also divisions if R is a field) used by an algorithm. In our analyses, the word *addition* stands for *addition or subtraction*; we do not count the latter separately. The number of other operations, such as index calculations or memory accesses, tends to be of the same order of magnitude. These are usually performed with machine instructions on single words, and their cost is negligible when the arithmetic quantities are large, say multiprecision integers. The input size is the number of ring elements that the input occupies. If the coefficients are integers or polynomials themselves, we may then consider separately the size of the coefficients involved and the cost for coefficient arithmetic.

We try to provide explicit (but not necessarily minimal) constants for the dominant term in our analyses of algorithms on polynomials when the cost measure is the number of arithmetic operations in the coefficient ring, but confine ourselves to O-estimates when counting the number of word operations for algorithms working on integers or polynomials with integral coefficients.

2.3. Multiplication

Following our program, we first consider the product $c = a \cdot b = \sum_{0 \le k \le n+m} c_k x^k$ of two polynomials a and b in $R[x]$, as in (3). Its coefficients are

$$c_k = \sum_{\substack{0 \le i \le n \\ 0 \le j \le m \\ i+j=k}} a_i b_j \tag{4}$$

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

for $0 \le k \le n+m$. We can just take this formula and turn it into a subroutine, after figuring out suitable loop variables and boundaries:

for $k = 0, \ldots, n+m$ **do**
  $c_k \longleftarrow 0$
  for $i = \max\{0, k-m\}, \ldots, \min\{n, k\}$ **do**
    $c_k \longleftarrow c_k + a_i \cdot b_{k-i}$

There are other ways to organize the loops. We learned in school the following algorithm.

ALGORITHM 2.3 Multiplication of polynomials.
Input: The coefficients of $a = \sum_{0 \le i \le n} a_i x^i$ and $b = \sum_{0 \le i \le m} b_i x^i$ in $R[x]$, where R is a (commutative) ring.
Output: The coefficients of $c = a \cdot b \in R[x]$.

1. **for** $i = 0, \ldots, n$ **do** $d_i \longleftarrow a_i x^i \cdot b$
2. **return** $c = \sum_{0 \le i \le n} d_i$

The multiplication $a_i x^i \cdot b$ is realized as the multiplication of each b_j by a_i plus a shift by i places. The variable x serves us just as a convenient way to write polynomials, and there is no arithmetic involved in "multiplying" by x or any power of it. Here is a small example:

$$
\begin{array}{rr}
5x^2+2x+1 \cdot & 2x^3 \quad +x^2 \quad +3x+5 \\
\hline
 & 2x^3 \quad +x^2 \quad +3x+5 \\
 & +4x^4 \quad +2x^3 \quad +6x^2+10x \\
 & +10x^5+5x^4+15x^3+25x^2 \\
\hline
 & 10x^5+9x^4+19x^3+32x^2+13x+5
\end{array}
$$

How much time does this take, that is, how many operations in the ground ring R? Each of the $n+1$ coefficients of a has to be multiplied with each of the $m+1$ coefficients of b, for a total of $(n+1)(m+1)$ multiplications. Then these are summed up in $n+m+1$ sums; summing s items costs $s-1$ additions. So the total number of additions is

$$(n+1)(m+1) - (n+m+1) = nm,$$

and the total cost for multiplication is $2nm+n+m+1 \le 2(n+1)(m+1)$ operations in R. (If a is monic, then the bound drops to $2nm+n \le 2n(m+1)$.) Thus we can say that two polynomials of degree at most n can be multiplied using $2n^2+2n+1$ operations, or $2n^2+O(n)$ operations, or $O(n^2)$ operations,

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

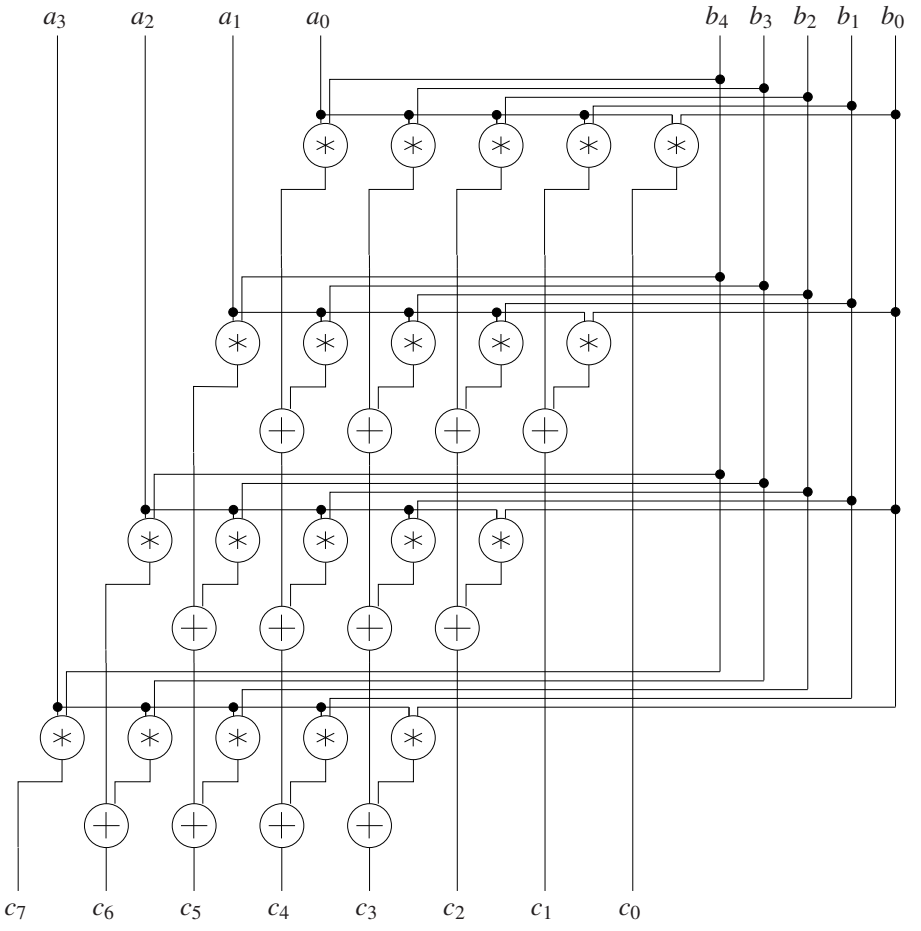


FIGURE 2.1: An arithmetic circuit for polynomial multiplication. The flow of control is directed downwards. An "electrical" view is to think of the edges as lines, where ring elements "flow", with "contact" crossings marked with a •, and no contact at the other crossings. The size of this circuit equals 32, the number of arithmetic gates in it.

or in **quadratic time**. The three expressions for the running time get progressively simpler but also less precise. In this book, each of the three versions has its place (and there are even more versions).

For a computer implementation, Algorithm 2.3 has the drawback of requiring us to store $n+1$ polynomials with $m+1$ coefficients each. A way around this is to interleave the final addition with the computation of the $a_i x^i b$. This takes the same time but uses only $O(n+m)$ storage and is shown in Figure 2.1 for $n=3$ and $m=4$. Each horizontal level corresponds to one pass through the loop body in step 1.

We will call **classical** those algorithms that take a definition of a function and implement it fairly literally, as the multiplication algorithms above implements the

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

formula (4). One might think that this is the only way of doing it. Fortunately, there are much faster ways of multiplying, in almost linear rather than quadratic time. We will study these fast algorithms in Part II. By contrast, for the addition problem no improvement is possible, nor is it necessary: the algorithm uses only linear time.

According to our general program, we now examine the integer case. The product of two single precision integers a, b between 0 and $2^{64}-1$ has "double precision": it lies in the interval $\{0,\dots,2^{128}-2^{65}+1\}$. We assume that our processor has a single precision multiplication instruction which returns the product in two 64-bit words c, d such that $a \cdot b = d \cdot 2^{64} + c$. Here is the integer analog of Algorithm 2.3.

ALGORITHM 2.4 Multiplication of multiprecision integers.
Input: Multiprecision integers $a = (-1)^s \sum_{0\le i\le n} a_i 2^{64i}$, $b = (-1)^t \sum_{0\le i\le m} b_i 2^{64i}$, not necessarily in standard representation, with $s,t \in \{0,1\}$.
Output: The multiprecision integer ab.

1. **for** $i = 0,\dots,n$ **do** $d_i \longleftarrow a_i 2^{64i} \cdot |b|$
2. **return** $c = (-1)^{s+t} \sum_{0\le i\le n} d_i$

Besides the multiplication by 2^{64i}, which is just a shift in the 2^{64}-ary representation, multiplication of a multiprecision integer b by a single precision integer a_i must be implemented. The time for this is $O(m)$ (Exercise 2.5), and the total time is quadratic: $O(nm)$. In line with our general program, we omit the details for implementing this efficiently.

We conclude this section with the example multiplication of $a = 521 = 5 \cdot 10^2 + 2 \cdot 10 + 1$ and $b = 2135 = 2 \cdot 10^3 + 10^2 + 3 \cdot 10 + 5$ in decimal representation, according to Algorithm 2.4.

$$\begin{array}{rr} 521 \cdot & 2135 \\ \hline & 2135 \\ & +42700 \\ & +1067500 \\ \hline & 1112335 \end{array}$$

2.4. Division with remainder

In many applications, calculations "modulo an integer" play an important role; examples from program checking, database integrity, coding theory and cryptography are discussed later in the book. But even when one is really only interested in integer problems, a "modular" approach is often computationally successful; we will see this for gcds and factorization of integer polynomials.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

The basic tool for modular arithmetic is **division with remainder**: given integers a, b, with b nonzero, we want to find a **quotient** q and a **remainder** r—both integers—so that

$$a = qb + r, \quad |r| < |b|.$$

In line with our general program, we first discuss the computational aspect of this problem for polynomials. So we are given $a, b \in R[x]$, with b nonzero, and want to find $q, r \in R[x]$ so that

$$a = qb + r, \quad \deg r < \deg b. \tag{5}$$

A first problem is that such q and r do not always exist: it is impossible to divide x^2 by $2x+1$ with remainder in $\mathbb{Z}[x]$! (See Exercise 2.8.) There is a way around this, the **pseudodivision** explained in Section 6.12. However, for the moment we simplify the problem by assuming that the leading coefficient $\mathrm{lc}(b)$ of b is a **unit** in R, so that it has an inverse $v \in R$ with $\mathrm{lc}(b)v = 1$. For $R = \mathbb{Z}$, that still only allows 1 or -1 as leading coefficient, but when R is a field, division with remainder by an arbitrary nonzero polynomial is possible.

We remind the reader of the "synthetic division" learned in high school with a small example in $\mathbb{Z}[x]$:

$$\begin{array}{rl}
3x^4+2x^3 & +x+5 : x^2+2x+3 = 3x^2-4x-1 \\
\cline{1-1}
-3x^4-6x^3-9x^2 & \\
\cline{1-1}
-4x^3-9x^2 & +x+5 \\
+4x^3+8x^2+12x & \\
\cline{1-1}
-x^2+13x+5 & \\
+x^2+2x+3 & \\
\cline{1-1}
15x+8 &
\end{array}$$

Thus the coefficients of the quotient $q = 3x^2 - 4x - 1$ are determined one by one, starting at the top, by setting them equal to the corresponding coefficient of the current "remainder" (in general, one additionally has to divide by $\mathrm{lc}(b)$), which initially is $a = 3x^4 + 2x^3 + x + 5$. Then the remainder is adjusted by subtracting the appropriate multiple of $b = x^2 + 2x + 3$. The final remainder is $r = 15x + 8$. The degree of q is $\deg a - \deg b$ if $q \neq 0$. The following algorithm formalizes this familiar *classical method* for division with remainder by a polynomial whose leading coefficient is a unit.

ALGORITHM 2.5 Polynomial division with remainder.
Input: $a = \sum_{0 \le i \le n} a_i x^i, b = \sum_{0 \le i \le m} b_i x^i \in R[x]$, with all $a_i, b_i \in R$, where R is a ring (commutative, with 1), b_m a unit, and $n \ge m \ge 0$.
Output: $q, r \in R[x]$ with $a = qb + r$ and $\deg r < m$.

1. $r \longleftarrow a, \quad u \longleftarrow b_m^{-1}$

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

2. **for** $i = n-m, n-m-1, \ldots, 0$ **do**

3. **if** $\deg r = m+i$ **then** $q_i \longleftarrow \mathrm{lc}(r)u, \quad r \longleftarrow r - q_i x^i b$
else $q_i \longleftarrow 0$

4. **return** $q = \sum_{0 \le i \le n-m} q_i x^i$ and r

Figure 2.2 represents this algorithm for $n = 7$ and $m = 4$ when $b_m = 1$. Each horizontal level in the circuit corresponds to one pass through step 3.

As in the polynomial multiplication algorithm, the multiplication $q_i x^i b$ in step 3 is just a multiplication of each b_j by q_i, followed by a shift by i places. The $(m+i)$th coefficient of r becomes 0 in step 3 and hence need not be computed. Thus the cost for one execution of step 3 is $m+1$ multiplications and m additions

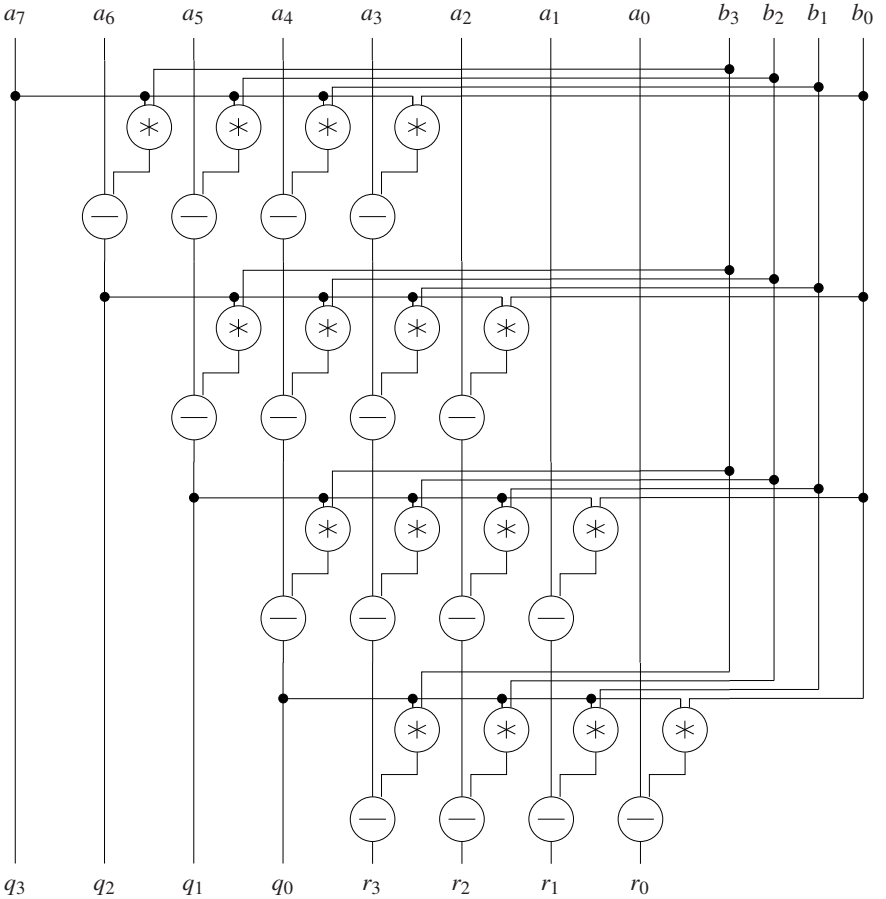


FIGURE 2.2: An arithmetic circuit for polynomial division. A subtraction node computes the difference of its left input minus its right input.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

in R if $\deg r = m + i$. Together, we have a cost of at most

$$(2m+1)(n-m+1) = (2\deg b + 1)(\deg q + 1) \in O(n^2)$$

additions and multiplications in R plus one division for inverting b_m, and only at most $2\deg b(\deg q + 1)$ additions and multiplications if b is monic. In many applications, we have $n < 2m$, and then the cost is at most $2m^2 + O(m)$ ring operations (plus an inversion), which is essentially the same as for multiplying two polynomials of degree at most m.

It is easy to see that the quotient and remainder are uniquely determined (when $\mathrm{lc}(b)$ is a unit). Namely, another equation $a = q^*b + r^*$, with $q^*, r^* \in R[x]$ and $\deg r^* < \deg b$, yields by subtraction

$$(q^* - q)b = r - r^*.$$

The right hand side has degree less than $\deg b$, and the left hand side has degree at least $\deg b$, unless $q^* - q = 0$. Therefore the latter is true, $q = q^*$, and $r = r^*$. We write "a quo b" for the quotient q and "a rem b" for the remainder r.

What about the integer case? The analog of Algorithm 2.5 is well known from high school, at least in the decimal representation:

$$\begin{array}{r} 32015 : 123 = 260 \\ \underline{-24600} \\ 7415 \\ \underline{-7380} \\ 35 \end{array}$$

The digits of the quotient are again determined one by one, starting at the top. At each step, one has to determine how often the leading digit of b divides the one or two leading digits of the current remainder. This requires a "double-by-single-precision" division instruction. However, it may happen that this "trial" quotient digit is too large, as in the first line: the quotient of 3 on division by 1 is 3, but the leading digit of the quotient $\lfloor 32015/123 \rfloor$ is 2.

Algorithmically, in each step, the product of the shifted divisor $2^{64i}b$ with a trial quotient digit is subtracted from some integer of the same length. If the result is negative, then the trial quotient digit is too large and one has to decrement it and add $2^{64i}b$ to the remainder (repeatedly, if necessary) until the remainder is nonnegative. If things are arranged properly, then one such step can be done with $O(\lambda(b))$ word operations. The length of the quotient—the number of iterations—is at most $\lambda(a) - \lambda(b) + 1$ (Exercise 2.7), and the overall cost estimate is again $O(\lambda(b)\lambda(q))$ or $O(m(n-m))$ word operations if a and b have lengths n and m, respectively. Due to the carries, the details are somewhat more complicated than in the polynomial case; the interested reader is referred to the comprehensive discussion in Knuth (1998), §4.3.1.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

In contrast to the polynomial case, the remainder r is not uniquely determined by the condition $|r| < |b|$: $13 = 2 \cdot 5 + 3 = 3 \cdot 5 + (-2)$. We will often follow the convention that the remainder be nonnegative and denote it by a rem b, and the corresponding quotient then is $\lfloor a/b \rfloor$ if $b > 0$.

Notes. Good texts on algorithms and their analysis are Brassard & Bratley (1996) and Cormen, Leiserson, Rivest & Stein (2009).

Addition and multiplication algorithms in decimal notation are explicitly described in Stevin (1585). Several algorithms for computer arithmetic, such as fast (carry look-ahead and carry-save) addition, are given in Cormen, Leiserson, Rivest & Stein (2009). For information about the highly active area of symbolic-numeric computations see the Special Issue of the *Journal of Symbolic Computation* (Watt & Stetter 1998) and Corless, Kaltofen & Watt (2003).

2.4. The first algorithm for division with remainder of polynomials appears in Nuñez (1567). He is, of course, limited by the concepts of his times to specific degrees, 3 and 1 in his case, and positive coefficients. On f°31r°, Nuñez writes: *Si el partidor fuere compuesto, partiremos las mayores dignidades de lo que se ha de partir por la mayor dignidad del partidor, dexandole en que pueda caber la otra dignidad del partidor, y lo q̃ viniere multiplicaremos por el partidor, y lo produzido por essa multiplicacion sacaremos de toda la sũma que se parte, y lo mismo obraremos en lo q̃ restare, por el modo q̃ tenemos quando partimos numero por numero. Y llegando a numero o dignidad en esta obra que sea de menor denominacion, que el partidor, quedara essa quantidad en quebrado, [...]*[1] He then explains the division of $12x^3 + 18x^2 + 27x + 17$ by $4x + 3$ with quotient $3x^2 + 2\frac{3}{4}x + 5\frac{3}{16}$ and remainder $1\frac{13}{16}$, and checks his result by multiplying out.

An anonymous author (1835) presents a decimal division algorithm for hand calculation based on a "10's complement" notation.

Exercises.

2.1 For an integer $r \in \mathbb{N}_{>1}$, we consider the variable-length radix r representation $(a_0, \ldots, a_{l-1})$ of a positive integer a, with $a = \sum_{0 \le i < l} a_i r^i$, $a_0, \ldots, a_{l-1} \in \{0, \ldots, r-1\}$, and $a_{l-1} \neq 0$. Prove that its length l is $\lfloor \log_r a \rfloor + 1$.

2.2 Design a representation for integers of unlimited size on a 64-bit machine.

2.3 (i) Specify a processor instruction analogous to the addition instruction mentioned in the text which performs subtraction of two single precision integers. Use the carry flag to indicate whether the result is negative or not.

(ii) Design an algorithm similar to Algorithm 2.1 for the subtraction of two multiprecision integers a and b of equal sign and with $|a| > |b|$.

(iii) Discuss how to decide whether $|a| > |b|$ holds.

2.4 Here is a piece of code implementing Algorithm 2.1 for nonnegative multiprecision integers (that is, when $s = 0$) on a hypothetical processor. Text enclosed in `/*` and `*/` is a comment. The

[1] If the divisor is composed [of more than one summand], we divide the leading term of the dividend by the leading term of the divisor, ignoring the other terms of the divisor, and we multiply the result by the divisor and subtract the result of this multiplication from the whole of the dividend, and we apply the same procedure to what is left, in the way we use it when we divide one number by another. And if we arrive in this procedure at numbers or terms whose degree is less than that of the divisor, then this quantity will remain as a fraction [...]

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

processor has 26 freely usable registers named A to Z. Initially, registers A and B point to the first word (the one containing the length) of the representations of a and b, respectively, and C points to a piece of memory where the representation of c shall be placed.

```
 1: LOAD N, [A]    /* load the word that A points to into register N */
 2: ADD K, N, 1    /* add 1 to register N and store the result in K
                      (without affecting the carry flag) */
 3: STORE [C], K   /* store K in the word that C points to */
 4: ADD A, A, 1    /* increase register A by 1 */
 5: ADD B, B, 1
 6: ADD C, C, 1
 7: LOAD I, 1      /* load the constant 1 into register I */
 8: CLEARC         /* clear carry flag */
 9: COMP I, N      /* compare the contents of registers I and N ...  */
10: BGT 20         /* ...  and jump to line 20 if I is greater */
11: LOAD S, [A]
12: LOAD T, [B]
13: ADDC S, S, T   /* add the contents of register T to register S
                      using the carry flag */
14: STORE [C], S
15: ADD A, A, 1
16: ADD B, B, 1
17: ADD C, C, 1
18: ADD I, I, 1
19: JMP 9          /* unconditionally jump to line 9 */
20: ADDC S, 0, 0   /* store carry flag in S */
21: STORE [C], S
22: RETURN
```

Suppose that our processor runs at 2 GHz and that the execution of one instruction takes one machine cycle = 0.5 nanoseconds $= 5 \cdot 10^{-10}$ seconds. Calculate the precise time, in terms of n, to run the above piece of code, and convince yourself that this is indeed $O(n)$.

2.5 Give an algorithm for multiplying a multiprecision integer b by a single precision integer a, making use of the single precision multiply instruction described in Section 2.3. Show that your algorithm uses $\lambda(b)$ single precision multiplications and the same number of single precision additions. Convert your algorithm into a machine program as in Exercise 2.4.

2.6 Prove that $\max\{\lambda(a),\lambda(b)\} \le \lambda(a+b) \le \max\{\lambda(a),\lambda(b)\}+1$ and $\lambda(a)+\lambda(b)-1 \le \lambda(ab) \le \lambda(a)+\lambda(b)$ hold for all $a,b \in \mathbb{N}_{>0}$.

2.7 Let $a > b \in \mathbb{N}_{>0}$, $m = \lambda(a)$, $n = \lambda(b)$ and $q = \lfloor a/b \rfloor$. Give tight upper and lower bounds for $\lambda(q)$ in terms of m and n.

2.8 Prove that in $\mathbb{Z}[x]$ one cannot divide x^2 by $2x+1$ with remainder as in (5).

2.9* Let R be an integral domain with field of fractions K and $a,b \in R[x]$ of degree $n \ge m \ge 0$. Then we can apply the polynomial division algorithm 2.5 to compute $q,r \in K[x]$ such that $a = qb+r$ and $\deg r < \deg b$.

(i) Prove that there exist $q,r \in R[x]$ with $a = qb+r$ and $\deg r < \deg b$ if and only if $\mathrm{lc}(b) \mid \mathrm{lc}(r)$ in R every time the algorithm passes through step 3, and that they are unique in that case.

(ii) Modify Algorithm 2.5 so that on input a,b, it decides whether $q,r \in R[x]$ as in (i) exist, and if so, computes them. Show that this takes the same number of operations in R as given in the text, where one operation is either an addition or a multiplication in R, or a test which decides whether an element $c \in R$ divides another element $d \in R$, and if so, computes the quotient $d/c \in R$.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

2.10 Let R be a ring (commutative, with 1) and $a = \sum_{0 \le i \le n} a_i x^i \in R[x]$ of degree n, with all $a_i \in R$. The **weight** $w(a)$ of a is the number of nonzero coefficients of a besides the leading coefficient:

$$w(a) = \#\{0 \le i < n : a_i \neq 0\}.$$

Thus $w(a) \le \deg a$, with equality if and only if all coefficients of a are nonzero. The **sparse** representation of a, which is particularly useful if a has small weight, is a list of pairs $(i, a_i)_{i \in I}$, with each $a_i \in R$ and $a = \sum_{i \in I} a_i x^i$. Then we can choose $\#I = w(a) + 1$.

(i) Show that two polynomials $a, b \in R[x]$ of weight $n = w(a)$ and $m = w(b)$ can be multiplied in the sparse representation using at most $2nm + n + m + 1$ arithmetic operations in R.

(ii) Draw an arithmetic circuit for division of a polynomial $a \in R[x]$ of degree less than 9 by $b = x^6 - 3x^4 + 2$ with remainder. Try to get its size as small as possible.

(iii) Let $n \ge m$. Show that quotient and remainder on division of a polynomial $a \in R[x]$ of degree less than n by $b \in R[x]$ of degree m, with $\mathrm{lc}(b)$ a unit, can be computed using $n - m$ divisions in R, and $w(b) \cdot (n - m)$ multiplications and subtractions in R each.

2.11 Let R be a ring and $k, m, n \in \mathbb{N}$. Show that the "classical" multiplication of two matrices $A \in R^{k \times m}$ and $B \in R^{m \times n}$ takes $(2m - 1)kn$ arithmetic operations in R.

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press

‘Immortality’ may be a silly word, but probably a mathematician
has the best chance of whatever it may mean.

Godfrey Harold Hardy (1940)

The ignoraunte multitude doeth, but as it was euer wonte, enuie that
knoweledge, whiche thei can not attaine, and wishe all men ignoraunt,
like unto themself. [. . .] Yea, the *pointe* in *Geometrie*,
and the unitie in *Arithmetike*, though bothe be undiuisible,
doe make greater woorkes, & increase greater multitudes,
then the brutishe bande of ignoraunce is hable to withstande.

Robert Recorde (1557)

If mathematics is considered to be a science
[that is, devoted to the description of nature and its laws],
it is more fundamental than any other.

Murray Gell-Mann (1994)

I have often wished, that I had employed about the speculative part of
geometry, and the cultivation of the specious Algebra [multivariate
polynomials] I had been taught very young, a good part of that time
and industry, that I had spent about surveying and fortification (of
which I remember I once wrote an entire treatise) and other practick
parts of mathematicks. And indeed the operations of symbolical
arithmetick (or the modern Algebra) seem to me to afford men one of
the clearest exercises of reason that I ever yet met with.

Robert Boyle (1671)

The length of this article will not be blamed by any one
who considers that, the sacred writers excepted, no Greek
has been so much read and so variously translated as Euclid.

Augustus De Morgan (c. 1844)

https://doi.org/10.1017/CBO9781139856065.004 Published online by Cambridge University Press